SOFTWARE AUTOMATION TEST DOCUMENT

Automatic Smart Pet Feeder — PawFeed Web Dashboard · Playwright Automation

1. Document Information

Field	Details
Project Name	Automatic Smart Pet Feeder
Application Name	PawFeed — Smart Pet Feeder Web Dashboard (with simulated feeder device)
Version	1.0.0
Test Cycle	Final Exam — Automation Cycle 1
Prepared By	Russel Carlo T. Manzon (IT233)
Reviewed By	
Date	October 17, 2026
Automation Framework	Playwright Test (@playwright/test v1.64.1), JavaScript
Test Environment	Local DEV — Windows 11 Pro, Node.js v23.8.0, Chromium, http://localhost:3000

2. Objective

Purpose

The objective of this automation testing effort is to turn the automatable scenarios from the *Define Automation Testing* document (AS-01 to AS-06) into real, repeatable Playwright tests that run against the Smart Pet Feeder web dashboard. The tests validate the feeder's key functions — manual feeding, portion accuracy, scheduled dispensing, low-food alerts, Wi-Fi reconnection and app-to-device schedule sync — reduce manual regression effort, and capture screenshot evidence of every run.

3. Scope

In Scope

- Manual Feed (Feed Now) and jam detection
- Portion size configuration and validation
- Scheduled dispensing using a mocked device clock
- Low Food alert notifications (simulated IR sensor)
- Wi-Fi drop and automatic reconnection (simulated network)

- App-to-device schedule synchronization time
- Device API validation through the web UI (status, feed, portion, schedules)

Out of Scope

- Physical / mechanical checks: anti-jam motor force (TC-05), cord safety (TC-11), lid-lock shake test (TC-13) — remain manual / hardware-in-the-loop
- Performance / load testing and penetration testing
- Native mobile app testing and real feeder firmware / hardware
- Third-party systems (cloud push-notification providers, Wi-Fi routers)

4. Test Environment

Component	Details
Environment	DEV (local machine)
Application URL	http://localhost:3000 (started automatically by Playwright `webServer`)
Database	None — device state is kept in memory by the Node.js server and reset before every test
Browser	Chromium (Playwright build, Chrome Headless Shell 156), viewport 1280 × 900
Mobile Device	N/A — web dashboard tested at desktop viewport
Operating System	Windows 11 Pro (10.0.26300)
API Endpoint	http://localhost:3000/api (status, feed, portion, schedules, sim/*)
Build Version	PawFeed 1.0.0 · Node.js v23.8.0 · Playwright 1.64.1

5. Automation Framework Details

Framework Type

☐ Data Driven

☐ Keyword Driven

☑ **Hybrid Framework** — Playwright Test runner + reusable helper functions (`setPortion`, `lastWeight`, `snap`), stable `data-testid` locators, and test data taken from the AS-01–AS-06 scenarios

☐ BDD Framework

☐ Model-Based Testing

Technology Stack

Component	Tool
Test Automation	**Playwright** (@playwright/test 1.64.1)

Component	Tool
Mobile Automation	N/A (web dashboard only)
API Automation	Playwright APIRequestContext (device reset before each test)
Programming Language	**JavaScript** (Node.js v23.8.0)
Build Tool	npm (scripts: `npm test`, `npm run report`)
Reporting	Playwright HTML Reporter (screenshots, video, trace) + JSON reporter
CI/CD	Not configured yet — tests run locally (see Section 16)
Source Control	Git (project folder ready to push; see Section 16)

Project Structure

File / Folder	Purpose
server.js	Node.js server + simulated feeder device API
public/index.html	PawFeed web dashboard (system under test)
tests/smart-pet-feeder.spec.js	The 9 automated test cases (ATC-01 – ATC-09)
tests/global-teardown.js	Stops the app server after the run
playwright.config.js	Runner config: browser, base URL, screenshots/video/trace, reporters
screenshots/	Screenshots saved by every test
playwright-report/	Generated HTML report

6. Entry Criteria

☑ Stable test environment available (local server starts automatically)
☑ Test data prepared (values from AS-01 – AS-06; device reset before each test)
☑ User stories completed (dashboard features for all six scenarios implemented)
☑ Test cases approved (based on the submitted Define Automation Testing document)
☑ Test scripts developed and reviewed
☑ Build deployed successfully (PawFeed 1.0.0 running on localhost:3000)

7. Exit Criteria

☑ All critical test cases executed (9/9)
☑ No Sev-1 or Sev-2 defects open
☑ Automation pass rate ≥ 95% (actual: 100%)
☑ Defect retesting completed (environment issues ENV-001 and ENV-002 re-tested after fix)

☐ Final test report approved (pending instructor review)

8. Automation Test Scenarios

Test ID	Module	Scenario	Maps to	Priority
AT-001	Manual Feed	Verify manual feed dispenses exactly one portion (~10 g)	AS-01 / TC-02	High
AT-002	Manual Feed	Verify manual feed is blocked when the chute is jammed	AS-01 / TC-02 (negative)	High
AT-003	Portion Settings	Verify 5-slot portion dispenses ~50 g and the setting persists	AS-02 / TC-03	High
AT-004	Portion Settings	Verify an invalid portion (99 slots) is rejected	AS-02 / TC-03 (negative)	High
AT-005	Scheduling	Verify a scheduled meal triggers dispensing on time	AS-03 / TC-04	High
AT-006	Alerts	Verify a Low Food alert is sent when hopper drops below 15%	AS-04 / TC-06	Medium
AT-007	Alerts	Verify no Low Food alert fires above the 15% threshold	AS-04 / TC-06 (negative)	Medium
AT-008	Connectivity	Verify the device auto-reconnects to Wi-Fi without manual setup	AS-05 / TC-08	High
AT-009	App Sync	Verify a schedule change syncs to the device in under 2 seconds	AS-06 / TC-14	Medium

9. Automated Test Cases

Each test case below was automated in Playwright and executed on October 17, 2026. The **Actual Result** values are recorded by the test script itself during the run, and every screenshot was taken automatically by the test at the moment its assertions passed.

ATC-01 — Verify manual feed dispenses exactly one portion (~10 g)

Field	Details
Test Case ID	ATC-01
Test Case Name	Verify manual feed dispenses exactly one portion (~10 g)
Module / Maps to	Manual Feed — AS-01 / TC-02
Priority	High
Preconditions	• Feeder is powered on and CONNECTED • Hopper food level is 100% • Portion size is 1 slot (default)

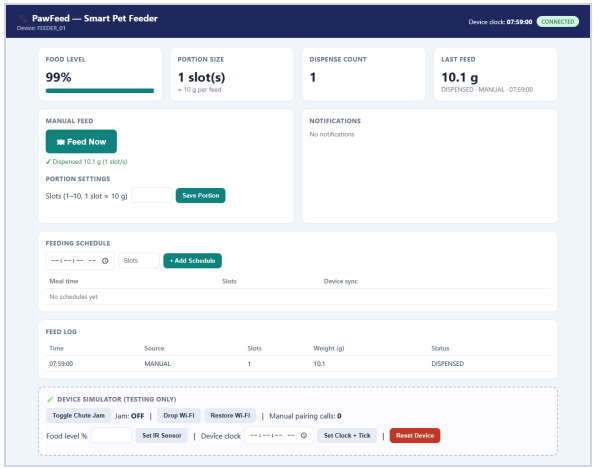
Test Data

Field	Value
Portion size	1 slot (≈10 g)
Tolerance	±5% → 9.5 g – 10.5 g

Test Steps

Step	Description	Expected Result
1	Reset device and open the dashboard	Dashboard loads, status CONNECTED, dispense count 0
2	Click “Feed Now”	Message “Dispensed … g (1 slot/s)” is shown
3	Read Dispense Count and Last Feed weight	Count = 1, weight within 9.5–10.5 g, one row in Feed Log

Field	Details
Expected Result	Exactly 1 dispense event is recorded and the weight is within ±5% of 10 g.
Actual Result	Dispense count went 0 → 1; dispensed 10.1 g (within 9.5–10.5 g).
Execution Result	**PASSED** (0.5 s)
Automation Status	☐ Not Started ☐ In Progress ☑ Automated ☑ Executed
Script	tests/smart-pet-feeder.spec.js — “ATC-01”



Screenshot: ATC-01_manual_feed_success.png (captured automatically by Playwright during the test)

ATC-02 — Verify manual feed is blocked when the chute is jammed

Field	Details
Test Case ID	ATC-02
Test Case Name	Verify manual feed is blocked when the chute is jammed
Module / Maps to	Manual Feed — AS-01 / TC-02 (negative)
Priority	High
Preconditions	• Feeder is powered on and CONNECTED • Chute jam can be simulated

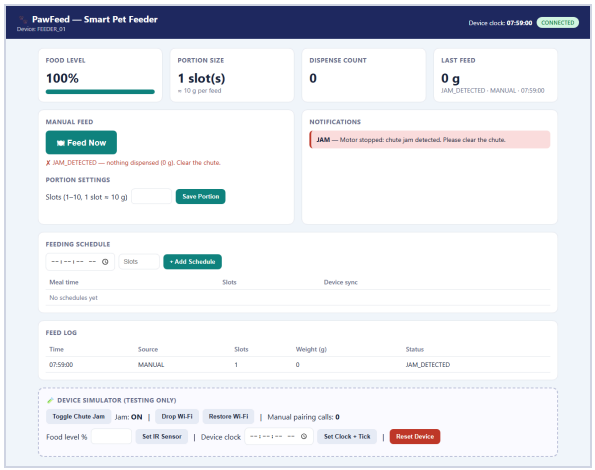
Test Data

Field	Value
Chute jam	ON
Expected status	JAM_DETECTED

Test Steps

Step	Description	Expected Result
1	Click “Toggle Chute Jam” in the Device Simulator	Jam indicator shows ON
2	Click “Feed Now”	Message “JAM_DETECTED — nothing dispensed (0 g)”
3	Check Last Feed, Dispense Count and Notifications	Weight 0 g, count stays 0, JAM notification visible

Field	Details
Expected Result	Feeder reports JAM_DETECTED, dispenses 0 g and alerts the owner.
Actual Result	Status JAM_DETECTED, 0 g dispensed, dispense count stayed 0, JAM notification shown.
Execution Result	**PASSED** (0.5 s)
Automation Status	☐ Not Started ☐ In Progress ☑ Automated ☑ Executed
Script	tests/smart-pet-feeder.spec.js — “ATC-02”



Screenshot: ATC-02_manual_feed_jam_detected.png (captured automatically by Playwright during the test)

ATC-03 — Verify 5-slot portion dispenses ~50 g and the setting persists

Field	Details
Test Case ID	ATC-03
Test Case Name	Verify 5-slot portion dispenses ~50 g and the setting persists
Module / Maps to	Portion Settings — AS-02 / TC-03
Priority	High
Preconditions	• App is open and paired with the device

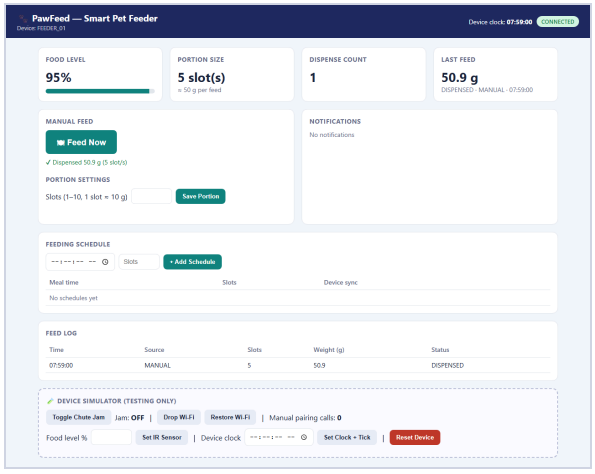
Test Data

Field	Value
Portion size	5 slots (≈50 g)
Tolerance	±5% → 47.5 g – 52.5 g

Test Steps

Step	Description	Expected Result
1	Enter 5 in the Slots field and click “Save Portion”	“Portion saved: 5 slot(s)” message
2	Reload the page	Portion Size still shows 5 slot(s)
3	Click “Feed Now” and read Last Feed weight	Weight within 47.5–52.5 g

Field	Details
Expected Result	Portion is saved on the device and the dispensed weight is within ±5% of 50 g.
Actual Result	Portion saved and still 5 slots after page reload; dispensed 50.9 g (within 47.5–52.5 g).
Execution Result	**PASSED** (0.6 s)
Automation Status	☐ Not Started ☐ In Progress ☑ Automated ☑ Executed
Script	tests/smart-pet-feeder.spec.js — “ATC-03”



Screenshot: ATC-03_portion_5_slots_50g.png (captured automatically by Playwright during the test)

ATC-04 — Verify an invalid portion (99 slots) is rejected

Field	Details
Test Case ID	ATC-04
Test Case Name	Verify an invalid portion (99 slots) is rejected
Module / Maps to	Portion Settings — AS-02 / TC-03 (negative)
Priority	High
Preconditions	• App is open and paired with the device • Current portion is 1 slot

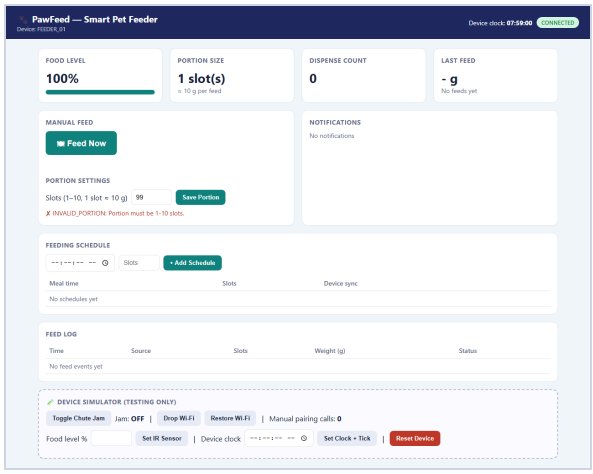
Test Data

Field	Value
Portion size	99 slots (exceeds 1–10 limit)
Expected status	INVALID_PORTION

Test Steps

Step	Description	Expected Result
1	Enter 99 in the Slots field and click “Save Portion”	Error “INVALID_PORTION: Portion must be 1-10 slots.”
2	Check Portion Size and Dispense Count	Portion stays 1 slot, nothing dispensed

Field	Details
Expected Result	The value is rejected with INVALID_PORTION and is not saved or dispensed.
Actual Result	App showed "INVALID_PORTION: Portion must be 1-10 slots."; portion stayed at 1 slot.
Execution Result	**PASSED** (0.5 s)
Automation Status	☐ Not Started ☐ In Progress ☑ Automated ☑ Executed
Script	tests/smart-pet-feeder.spec.js — “ATC-04”



Screenshot: ATC-04_invalid_portion_rejected.png (captured automatically by Playwright during the test)

ATC-05 — Verify a scheduled meal triggers dispensing on time

Field	Details
Test Case ID	ATC-05
Test Case Name	Verify a scheduled meal triggers dispensing on time
Module / Maps to	Scheduling — AS-03 / TC-04
Priority	High
Preconditions	• Device clock is synced (starts at 07:59:00) • Schedule can be created in the app

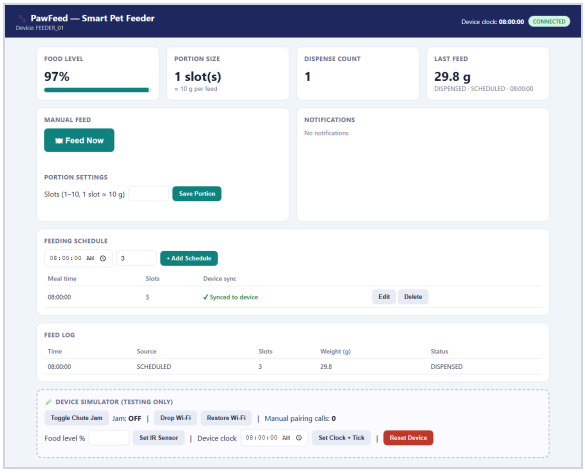
Test Data

Field	Value
Meal time	08:00:00
Portion	3 slots (≈30 g)
Time window	±1 minute

Test Steps

Step	Description	Expected Result
1	Add a schedule for 08:00:00 with 3 slots	Row appears and shows “Synced to device”
2	Set the device clock to 08:00:00 and tick (mocked clock)	Device clock shows 08:00:00
3	Check the Feed Log	New SCHEDULED / DISPENSED row within ±60 s of 08:00:00, ≈30 g

Field	Details
Expected Result	A scheduled feed event is logged within ±1 minute of the scheduled time.
Actual Result	SCHEDULED feed logged at 08:00:00 (0 s from target, within ±60 s); dispensed 29.8 g for 3 slots.
Execution Result	**PASSED** (1.9 s)
Automation Status	☐ Not Started ☐ In Progress ☑ Automated ☑ Executed
Script	tests/smart-pet-feeder.spec.js — “ATC-05”



Screenshot: ATC-05_scheduled_feed_triggered.png (captured automatically by Playwright during the test)

ATC-06 — Verify a Low Food alert is sent when hopper drops below 15%

Field	Details
Test Case ID	ATC-06
Test Case Name	Verify a Low Food alert is sent when hopper drops below 15%
Module / Maps to	Alerts — AS-04 / TC-06
Priority	Medium
Preconditions	• IR sensor reading can be simulated

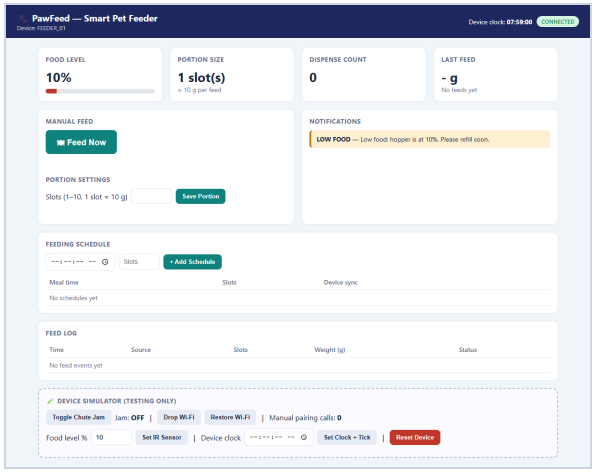
Test Data

Field	Value
Food level	10%
Alert threshold	< 15%

Test Steps

Step	Description	Expected Result
1	Enter 10 in “Food level %” and click “Set IR Sensor”	Food Level shows 10% (red bar)
2	Watch the Notifications panel (max 3 s)	“LOW FOOD” notification appears

Field	Details
Expected Result	The app receives a Low Food notification within a few seconds.
Actual Result	Food level showed 10% (red bar) and a LOW FOOD notification appeared.
Execution Result	**PASSED** (0.5 s)
Automation Status	☐ Not Started ☐ In Progress ☑ Automated ☑ Executed
Script	tests/smart-pet-feeder.spec.js — “ATC-06”



Screenshot: ATC-06_low_food_alert_10_percent.png (captured automatically by Playwright during the test)

ATC-07 — Verify no Low Food alert fires above the 15% threshold

Field	Details
Test Case ID	ATC-07
Test Case Name	Verify no Low Food alert fires above the 15% threshold
Module / Maps to	Alerts — AS-04 / TC-06 (negative)
Priority	Medium
Preconditions	• IR sensor reading can be simulated

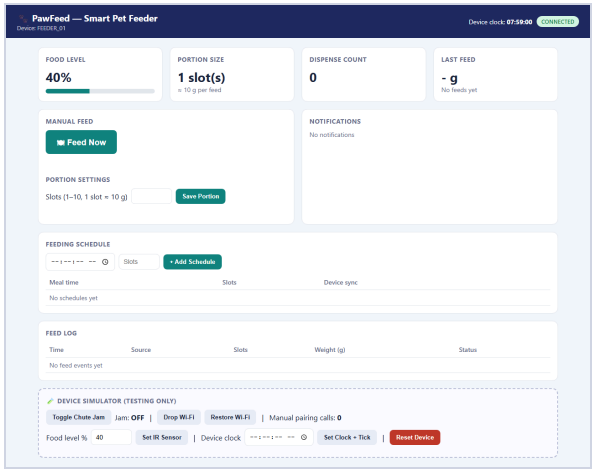
Test Data

Field	Value
Food level	40%
Alert threshold	< 15%

Test Steps

Step	Description	Expected Result
1	Enter 40 in “Food level %” and click “Set IR Sensor”	Food Level shows 40%
2	Wait 1.5 s and check Notifications	No LOW FOOD notification

Field	Details
Expected Result	No Low Food alert is shown when the hopper is above 15%.
Actual Result	Food level showed 40%; no LOW FOOD notification after waiting 1.5 s.
Execution Result	**PASSED** (1.9 s)
Automation Status	☐ Not Started ☐ In Progress ☑ Automated ☑ Executed
Script	tests/smart-pet-feeder.spec.js — “ATC-07”



Screenshot: ATC-07_no_alert_40_percent.png (captured automatically by Playwright during the test)

ATC-08 — Verify the device auto-reconnects to Wi-Fi without manual setup

Field	Details
Test Case ID	ATC-08
Test Case Name	Verify the device auto-reconnects to Wi-Fi without manual setup
Module / Maps to	Connectivity — AS-05 / TC-08
Priority	High
Preconditions	• Device was previously paired and CONNECTED

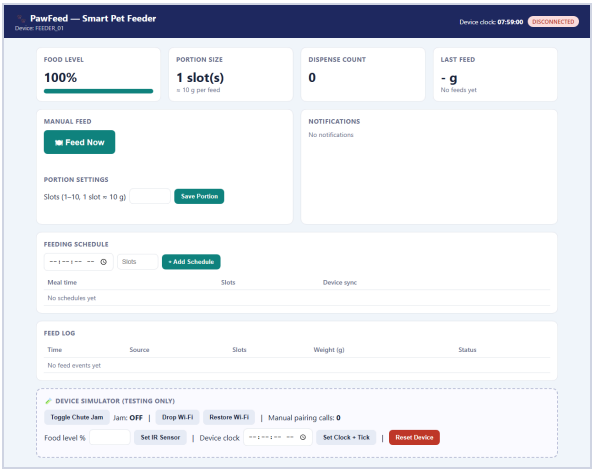
Test Data

Field	Value
Outage length	5 s
Reconnect timeout	60 s
Allowed pairing calls	0

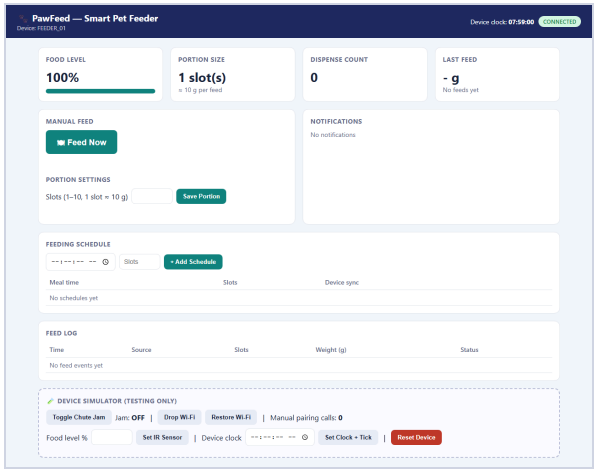
Test Steps

Step	Description	Expected Result
1	Click “Drop Wi-Fi”	Status badge shows DISCONNECTED
2	Wait 5 s, then click “Restore Wi-Fi”	Status shows RECONNECTING
3	Poll the status badge (up to 60 s)	Status returns to CONNECTED
4	Check “Manual pairing calls”	Value is 0

Field	Details
Expected Result	Status returns to CONNECTED within 60 s with no manual pairing call.
Actual Result	Status went DISCONNECTED → RECONNECTING → CONNECTED in 3.4 s after Wi-Fi was restored; manual pairing calls = 0.
Execution Result	**PASSED** (8.9 s)
Automation Status	☐ Not Started ☐ In Progress ☑ Automated ☑ Executed
Script	tests/smart-pet-feeder.spec.js — “ATC-08”



Screenshot: ATC-08a_wifi_dropped.png (captured automatically by Playwright during the test)



Screenshot: ATC-08b_wifi_reconnected.png (captured automatically by Playwright during the test)

ATC-09 — Verify a schedule change syncs to the device in under 2 seconds

Field	Details
Test Case ID	ATC-09
Test Case Name	Verify a schedule change syncs to the device in under 2 seconds
Module / Maps to	App Sync — AS-06 / TC-14
Priority	Medium
Preconditions	• App and device are online and paired • One schedule exists (08:00:00)

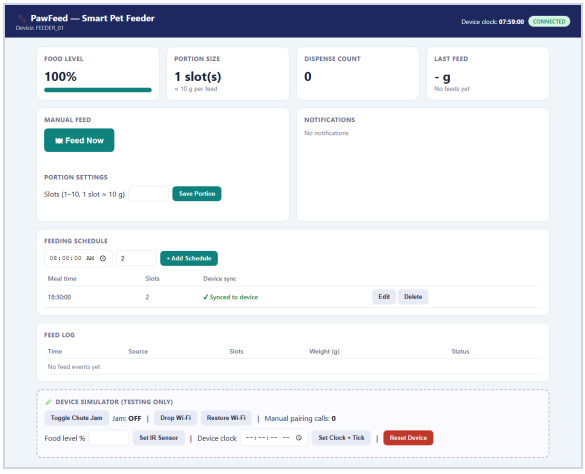
Test Data

Field	Value
Original time	08:00:00
New time	18:30:00
Sync limit	< 2 s

Test Steps

Step	Description	Expected Result
1	Add a schedule at 08:00:00 and wait for “Synced to device”	Schedule synced
2	Click “Edit”, change the time to 18:30:00 and start the timer on “Save”	Row shows 18:30:00, “Syncing…”
3	Wait for “Synced to device” and stop the timer	Elapsed time under 2 s

Field	Details
Expected Result	The device schedule matches the app’s new value in under 2 seconds.
Actual Result	Schedule changed 08:00:00 → 18:30:00 and showed "Synced to device" after 0.50 s (limit < 2 s).
Execution Result	**PASSED** (1.8 s)
Automation Status	☐ Not Started ☐ In Progress ☑ Automated ☑ Executed
Script	tests/smart-pet-feeder.spec.js — “ATC-09”



Screenshot: ATC-09_schedule_sync_under_2s.png (captured automatically by Playwright during the test)

10. Test Data Management

Data Type	Source	Owner
Device state (food level, portion, schedules, logs)	Reset through `POST /api/sim/reset` in `beforeEach`	QA (Playwright hook)
Portion sizes, meal times, thresholds	Hard-coded in the spec from AS-01 – AS-06	QA Team
Sensor & hardware inputs (jam, IR food level, Wi-Fi, clock)	Device Simulator panel / `/api/sim/*`	QA Team
User accounts / API tokens	N/A — local build has no login	—

11. Defect Tracking

No product defects were found in the final run: all 9 test cases passed. Two test-environment issues were found while setting up the automation and were fixed and re-tested:

Defect ID	Summary	Severity	Status
ENV-001	Playwright took ~3 minutes to stop the app server on Windows after each run. Fixed by adding a global teardown that asks the server to shut down.	Low	Fixed
ENV-002	Food-level bar was captured mid-animation, so the screenshot showed a shorter bar than the % value. Fixed by removing the CSS transition.	Low	Fixed

Severity Classification: Critical · High · Medium · Low

12. Test Execution Summary

Metric	Value
Total Test Cases	**9**
Automated Test Cases	**9**
Executed Test Cases	**9**
Passed	**9**
Failed	**0**
Blocked / Skipped	**0**

Metric	Value
Pass Percentage	**100%**
Total Execution Time	**18.6 s**
Run Date / Time	**October 17, 2026 at 7:57:41 AM**

Formula: Pass Rate (%) = (Passed / Executed) × 100 = (9 / 9) × 100 = **100%**

Execution Evidence

Screenshots taken by the tester during the run on October 17, 2026 at 7:57:41 AM.

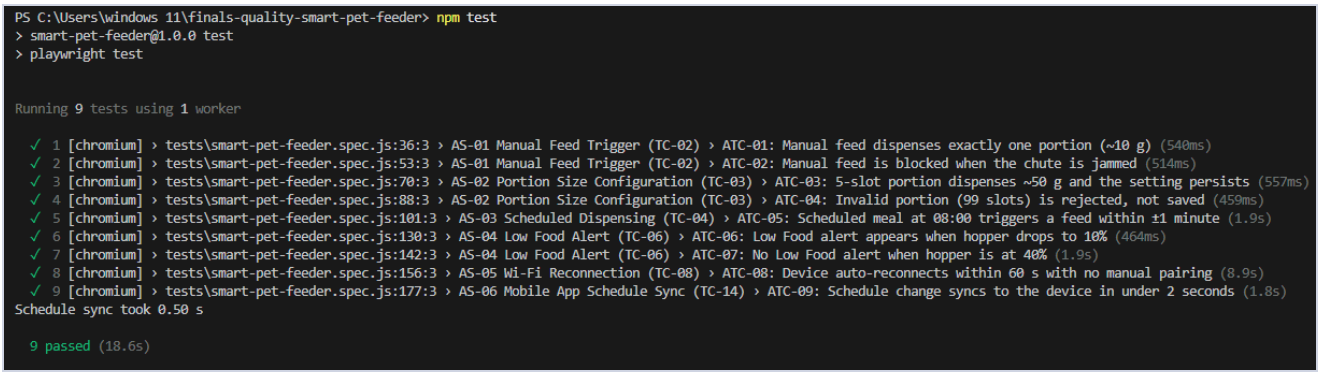


Figure 1 — Terminal: `npm test` (playwright test) — 9 passed in 18.6 s

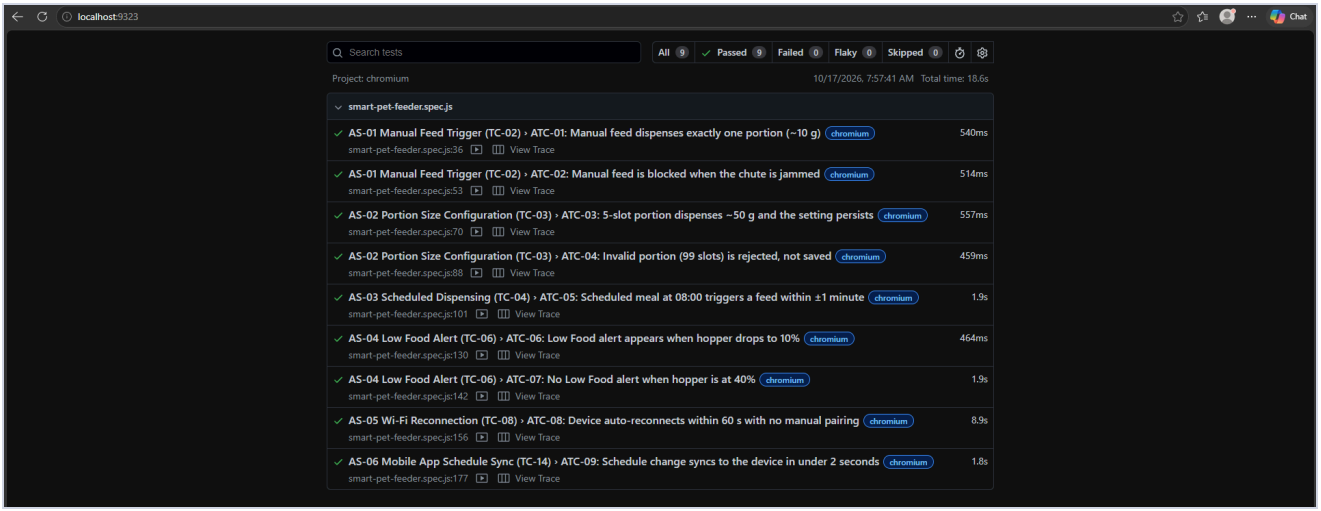


Figure 2 — Playwright HTML report (`npx playwright show-report`, http://localhost:9323): All 9 · Passed 9 · Failed 0 · Flaky 0 · Skipped 0

13. Automation Coverage

Module	Total TC	Automated	Coverage %
Manual Feed (TC-02)	2	2	100%

Module	Total TC	Automated	Coverage %
Portion Settings (TC-03)	2	2	100%
Scheduling (TC-04)	1	1	100%
Low Food Alerts (TC-06)	2	2	100%
Wi-Fi Connectivity (TC-08)	1	1	100%
App Schedule Sync (TC-14)	1	1	100%
Physical / Hardware (TC-05, TC-11, TC-13)	3	0	0% (manual only)
Total	**12**	**9**	**75%**

> ATC-02 covers the software side of TC-05 (jam is detected and the motor stops). The physical anti-jam force, cord integrity and lid-lock shake test need real hardware, so they stay manual as stated in the Define Automation Testing document.

14. Risks and Assumptions

Risks

- The device is simulated; real feeder hardware may behave differently (motor timing, sensor noise).
- Timing-based tests (Wi-Fi reconnect, sync < 2 s) could become flaky on a slow machine.
- Dispensed weight has random ±2% variance, so weight checks rely on the ±5% tolerance.
- Requirement changes in the original Test Plan would need the scripts updated.

Assumptions

- Port 3000 is free and Node.js + Playwright Chromium are installed.
- The requirements in the Test Plan / Define Automation Testing document are final.
- The device API is reachable during execution; tests run one at a time (1 worker) because they share one device.

15. Reporting and Dashboard

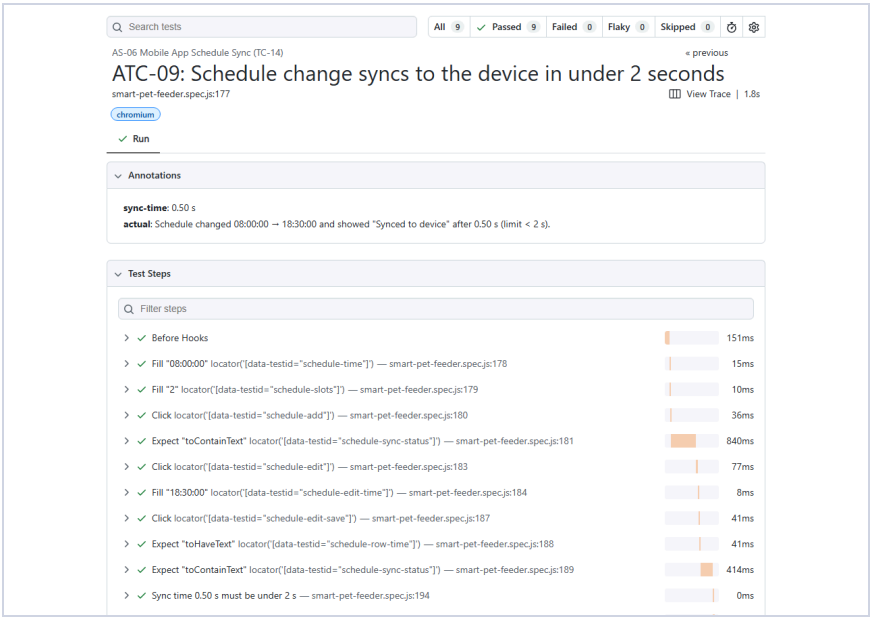
Daily Metrics (from the Playwright HTML report)

- Build Status: PawFeed 1.0.0 — app starts automatically
- Execution Status: 9/9 passed
- Defect Trend: 0 product defects; 2 environment issues fixed
- Automation Coverage: 9 of 12 test cases (75%)
- Failed Tests: 0
- Execution Duration: 18.6 s

Reports Used

- **Playwright HTML Report** (`npm run report`) — per-test steps, screenshots, video and trace

- Playwright JSON report (`docs/results.json`) — used to fill in this document



Playwright HTML report — step-by-step detail of ATC-09 with the recorded sync time

16. CI/CD Integration Checklist

Item	Status	Notes
Source Code in Git	☐	Project is ready to push to GitHub
Build Pipeline Configured	☐	Not yet — planned GitHub Actions workflow
Automated Execution Triggered	☑	One command (`npm test`) starts the app and runs all tests
Test Reports Published	☑	HTML + JSON reports generated on every run
Notifications Enabled	☐	Not yet
Quality Gates Configured	☑	Run exits with an error code if any test fails

17. Test Sign-Off

Role	Name	Signature	Date
QA Lead	Russel Carlo T. Manzon		October 17, 2026
Test Manager			
Development Lead	Russel Carlo T. Manzon		October 17, 2026
Business Owner			

Final Recommendation

Go Live Status:

☐ Go
☑ **Conditional Go** — all 9 automated software tests passed (100%). Go-live is conditional on completing the manual hardware tests (TC-05 anti-jam force, TC-11 cord safety, TC-13 lid lock).
☐ No-Go